

Output Control Structures

1.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
16 ;
17 INSERT INTO customers VALUES
18 (1, 'Ramesh Kulkarni', 65, 12000, 'N'),
19 (2, 'Sneha Patil', 45, 8000, 'N'),
20 (3, 'Arvind Joshi', 70, 15000, 'N'),
21 (4, 'Priya Deshmukh', 32, 5000, 'N');
22 INSERT INTO loans VALUES
23 (101, 1, 9.50, CURDATE() + INTERVAL 10 DAY),
24 (102, 2, 8.00, CURDATE() + INTERVAL 45 DAY),
25 (103, 3, 10.20, CURDATE() + INTERVAL 5 DAY),
26 (104, 4, 7.50, CURDATE() + INTERVAL 60 DAY);
27
28 SELECT * FROM customers;
29 SELECT * FROM loans;
30
31
```

The Result Grid shows the output of the queries:

customer_id	customer_name	age	balance	is_vip
1	Ramesh Kulkarni	65	12000.00	N
2	Sneha Patil	45	8000.00	N
3	Arvind Joshi	70	15000.00	N
4	Priya Deshmukh	32	5000.00	N

The Action Output shows the execution details:

#	Time	Action	Message	Duration / Fetch
2	16:57:44	USE bank_db		
3	16:58:04	CREATE TABLE customers (customer_id INT PRIMARY KEY, customer_name VARCHAR(100), a...	0 row(s) affected	0.140 sec
4	16:58:28	CREATE TABLE loans (loan_id INT PRIMARY KEY, customer_id INT, interest_rate DECIMAL(5,2...	0 row(s) affected	0.109 sec
5	16:58:47	INSERT INTO customers VALUES (1, 'Ramesh Kulkarni', 65, 12000, 'N'), (2, 'Sneha Patil', 45, 8000, 'N'), (...)	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
6	16:59:10	INSERT INTO loans VALUES (101, 1, 9.50, CURDATE() + INTERVAL 10 DAY), (102, 2, 8.00, CURDATE...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.078 sec
7	16:59:25	SELECT * FROM customers LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

2.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
22 INSERT INTO loans VALUES
23 (101, 1, 9.50, CURDATE() + INTERVAL 10 DAY),
24 (102, 2, 8.00, CURDATE() + INTERVAL 45 DAY),
25 (103, 3, 10.20, CURDATE() + INTERVAL 5 DAY),
26 (104, 4, 7.50, CURDATE() + INTERVAL 60 DAY);
27 SELECT * FROM customers;
28 SELECT * FROM loans;
29 UPDATE loans l
30 JOIN customers c
31 ON l.customer_id = c.customer_id
32 SET l.interest_rate = l.interest_rate * 0.99
33 WHERE c.age > 60;
34
35 SELECT * FROM loans;
36
37
```

The Result Grid shows the output of the queries:

loan_id	customer_id	interest_rate	due_date
101	1	9.41	2026-06-30
102	2	8.00	2026-08-04
103	3	10.10	2026-06-25
104	4	7.50	2026-08-19

The Action Output shows the execution details:

#	Time	Action	Message	Duration / Fetch
4	16:58:28	CREATE TABLE loans (loan_id INT PRIMARY KEY, customer_id INT, interest_rate DECIMAL(5,2...	0 row(s) affected	0.109 sec
5	16:58:47	INSERT INTO customers VALUES (1, 'Ramesh Kulkarni', 65, 12000, 'N'), (2, 'Sneha Patil', 45, 8000, 'N'), (...)	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
6	16:59:10	INSERT INTO loans VALUES (101, 1, 9.50, CURDATE() + INTERVAL 10 DAY), (102, 2, 8.00, CURDATE...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.078 sec
7	16:59:25	SELECT * FROM customers LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
8	17:00:10	UPDATE loans l JOIN customers c ON l.customer_id = c.customer_id SET l.interest_rate = l.interest_rate * ...	2 row(s) affected, 2 warnings: 1265 Data truncated for column 'interest_rate' at row 1 1265 Data truncate...	0.016 sec
9	17:00:26	SELECT * FROM loans LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

3.

The screenshot displays the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows a list of databases including 'attendance_system', 'bank', 'collection', 'college', 'collegedb', 'company', 'dairy_db', 'hotel_employee', 'it_44', 'lapdb', 'library_db', 'schooldb', 'sys', 'team_db', and 'ty_it_b'. The 'bank' database is selected, showing its tables, views, stored procedures, and functions.

The main editor shows a SQL query (Query 1) with the following code:

```

36 SELECT
37     CONCAT(
38         'Reminder: ',
39         c.customer_name,
40         ' has loan ',
41         l.loan_id,
42         ' due on ',
43         DATE_FORMAT(l.due_date, '%d-%b-%Y')
44     ) AS Reminder_Message
45 FROM loans l
46 JOIN customers c
47 ON l.customer_id = c.customer_id
48 WHERE l.due_date BETWEEN CURDATE()
49 AND CURDATE() + INTERVAL 30 DAY;
50
51 SELECT * FROM customers;

```

The 'Result Grid' shows the output of the query, displaying a table with columns: customer_id, customer_name, age, balance, and is_vip. The data is as follows:

customer_id	customer_name	age	balance	is_vip
1	Ramesh Kulkarni	65	12000.00	N
2	Sneha Patel	45	8000.00	N
3	Arvind Joshi	70	15000.00	N
4	Priya Deshmukh	32	5000.00	N

The 'Output' pane shows the execution log with the following actions:

#	Time	Action	Message	Duration / Fetch
6	16:59:10	INSERT INTO loans VALUES (101, 1, 9.50, CURDATE() + INTERVAL 10 DAY), (102, 2, 8.00, CURDATE() + INTERVAL 10 DAY), (103, 3, 7.50, CURDATE() + INTERVAL 10 DAY), (104, 4, 6.50, CURDATE() + INTERVAL 10 DAY);	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.078 sec
7	16:59:25	SELECT * FROM customers LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
8	17:00:10	UPDATE loans l JOIN customers c ON l.customer_id = c.customer_id SET l.interest_rate = l.interest_rate * 1.05;	2 row(s) affected, 2 warnings(s): 1265 Data truncated for column 'interest_rate' at row 1 1265 Data truncated for column 'interest_rate' at row 2	0.016 sec
9	17:00:26	SELECT * FROM loans LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
10	17:01:12	SELECT CONCAT('Reminder: ', c.customer_name, ' has loan ', l.loan_id, ' due on ', DATE_FORMAT(l.due_date, '%d-%b-%Y')) AS Reminder_Message FROM loans l JOIN customers c ON l.customer_id = c.customer_id WHERE l.due_date BETWEEN CURDATE() AND CURDATE() + INTERVAL 30 DAY;	2 row(s) returned	0.016 sec / 0.000 sec
11	17:01:33	SELECT * FROM customers LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

The bottom status bar shows the system temperature as 35°C, weather as 'Partly sunny', and the date and time as 17:01 on 20-06-2026.